

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

Aim

To implement audio convolution and inverse filtering using different impulse responses in Python, generate processed audio signals, compare the original, convolved, and restored signals, and analyze the effectiveness of inverse filtering in reducing or suppressing background music.

Objectives

1. To understand the concept of convolution in Digital Signal Processing (DSP).
2. To study the role of impulse responses in audio signal processing.
3. To implement convolution using different impulse responses.
4. To perform inverse filtering to reconstruct the original audio signal.
5. To compare the original, convolved, and inverse filtered signals.
6. To analyze the effect of different impulse responses on background music suppression.
7. To evaluate the effectiveness and limitations of inverse filtering.

Theory

Convolution is a mathematical operation widely used in Digital Signal Processing (DSP) to determine the output of a Linear Time-Invariant (LTI) system. It combines an input signal with the impulse response of a system to produce a modified output signal.

The convolution operation is expressed as

$$y(n) = x(n) * h(n)$$

where

- $x(n)$ = Input audio signal
- $h(n)$ = Impulse response
- $y(n)$ = Convolved output signal

An impulse response defines how a system reacts to a unit impulse. Different impulse responses simulate different audio effects such as smoothing, filtering, echo, and reverberation.

Inverse filtering is used to estimate the original signal from the convolved signal by reversing the effect of the impulse response. It is performed in the frequency domain using the Fourier Transform.

The inverse filtering equation is

$$X(f) = \frac{Y(f)}{H(f)}$$

where

- $X(f)$ = Restored signal
- $Y(f)$ = Fourier Transform of the convolved signal

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

- $H(f)$ = Fourier Transform of the impulse response

A small constant (ϵ) is added to avoid division by zero when the frequency response contains very small values.

Software Requirements

- NumPy
- SciPy
- Matplotlib
- Jupyter Notebook / VS Code

Algorithm

1. Import the required Python libraries.
2. Read the input WAV audio file.
3. Convert stereo audio into mono.
4. Normalize the audio signal.
5. Create folders for saving output files.
6. Define ten different impulse responses.
7. Apply convolution to the audio signal using each impulse response.
8. Normalize the convolved signal.
9. Compute the inverse filter in the frequency domain.
10. Apply inverse filtering to restore the audio signal.
11. Normalize the restored signal.
12. Save the convolved and restored audio files.
13. Plot the original, convolved, and inverse filtered waveforms.
14. Compare the results for different impulse responses.
15. Draw conclusions regarding background music suppression.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

Impulse Responses Used

S.No	Impulse Response	Purpose
1	Identity	No change to the signal
2	Noise Reduction	Smooths the signal and reduces noise
3	Low-Pass Filter	Removes high-frequency components
4	High-Pass Filter	Removes low-frequency components
5	Edge Detection	Highlights rapid changes
6	Sharpen	Enhances audio details
7	Room Reverb	Simulates a small room environment
8	Hall Reverb	Simulates a large hall environment
9	Echo	Produces delayed repetitions
10	Exponential Decay	Produces a gradually fading reverberation effect

Input

One WAV audio file containing speech and background music.

Code and Output:

```

import os
import numpy as np
import matplotlib.pyplot as plt

from scipy.io import wavfile
from scipy.signal import convolve
from scipy.fft import fft, ifft

```

[1] ✓ 2.7s

Generate + Code + Markdown Python

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

```

file_path = r"C:\Users\devah\Downloads\Matti-Tisava-Matti-Bommanu-Chesava.wav"

if not os.path.exists(file_path):
    raise FileNotFoundError("Audio file not found.")

fs, audio = wavfile.read(file_path)

if audio.ndim == 2:
    audio = np.mean(audio, axis=1)

audio = audio.astype(np.float32)

audio = audio / np.max(np.abs(audio))

audio_folder = "Processed_Audio"
graph_folder = "Graphs"

os.makedirs(audio_folder, exist_ok=True)
os.makedirs(graph_folder, exist_ok=True)

print("=" * 60)
print("Audio Loaded Successfully")
print("=" * 60)
print("Sampling Rate :", fs)
print("Total Samples :", len(audio))
print("Duration      :", round(len(audio) / fs, 2), "seconds")
print("Audio Folder   :", os.path.abspath(audio_folder))
print("Graph Folder    :", os.path.abspath(graph_folder))

```

[2] ✓ 0.5s Python

```

=====
Audio Loaded Successfully
=====
Sampling Rate : 48000
Total Samples : 19699200
Duration      : 410.4 seconds
Audio Folder  : c:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Processed_Audio
Graph Folder  : c:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Graphs

```

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
ir_identity = np.array([1.0])
ir_noise_reduction = np.ones(5) / 5

ir_lowpass = np.array([
    0.25,
    0.50,
    0.25
])

ir_highpass = np.array([
    -1,
    2,
    -1
])

ir_edge = np.array([
    1,
    0,
    -1
])

ir_sharpen = np.array([
    -0.5,
    2.0,
    -0.5
])

ir_room = np.array([
    1.0,
    0.6,
    0.3,
    0.15
])

ir_hall = np.exp(
    -np.linspace(0, 2, 25)
)

ir_echo = np.zeros(30)
ir_echo[0] = 1
ir_echo[10] = 0.6
ir_echo[20] = 0.4

ir_decay = np.exp(
    -np.linspace(0, 3, 15)
)

impulse_responses = {
    "Identity": ir_identity,
    "NoiseReduction": ir_noise_reduction,
    "LowPass": ir_lowpass,
    "HighPass": ir_highpass,
    "EdgeDetection": ir_edge,
    "Sharpen": ir_sharpen,
    "RoomReverb": ir_room,
    "HallReverb": ir_hall,
    "Echo": ir_echo,
    "Decay": ir_decay
}

print("Total Impulse Responses :", len(impulse_responses))

for name in impulse_responses:
    print(name)
```

Python

```
✓ 0.0s

Total Impulse Responses : 10
Identity
NoiseReduction
LowPass
HighPass
EdgeDetection
Sharpen
RoomReverb
HallReverb
Echo
Decay
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

```

name = "Identity"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "Identity_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "Identity_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[::100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[::100], color="red")
plt.title("Convolved Audio - Identity")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[::100], color="green")
plt.title("Inverse Filtered Audio - Identity")
plt.grid(True)

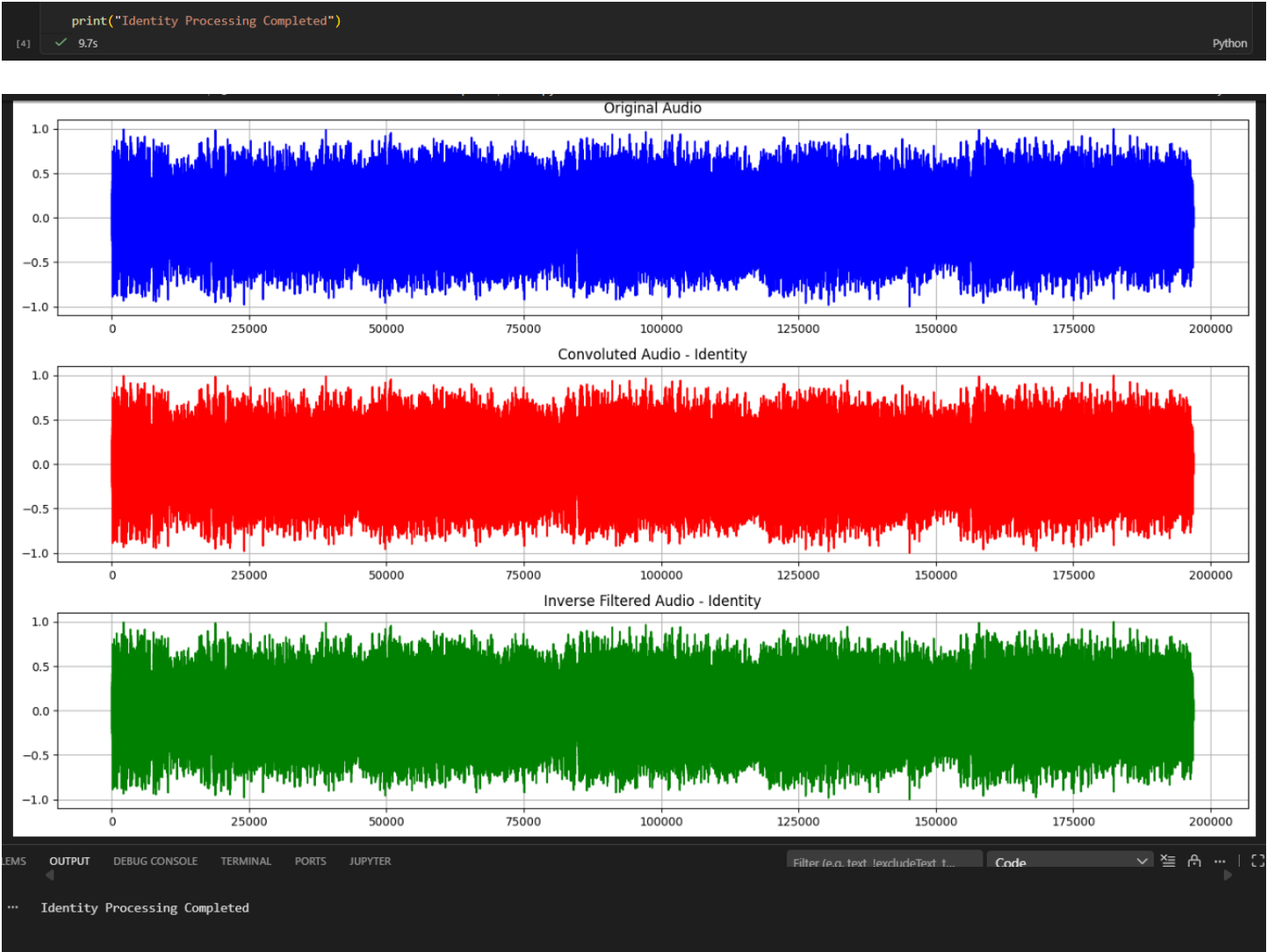
plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "Identity.png"
    )
)

plt.show()

```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147



Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "NoiseReduction"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "NoiseReduction_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "NoiseReduction_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Noise Reduction")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Noise Reduction")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "NoiseReduction.png"
    )
)

plt.show()

print("Noise Reduction Processing Completed")
```

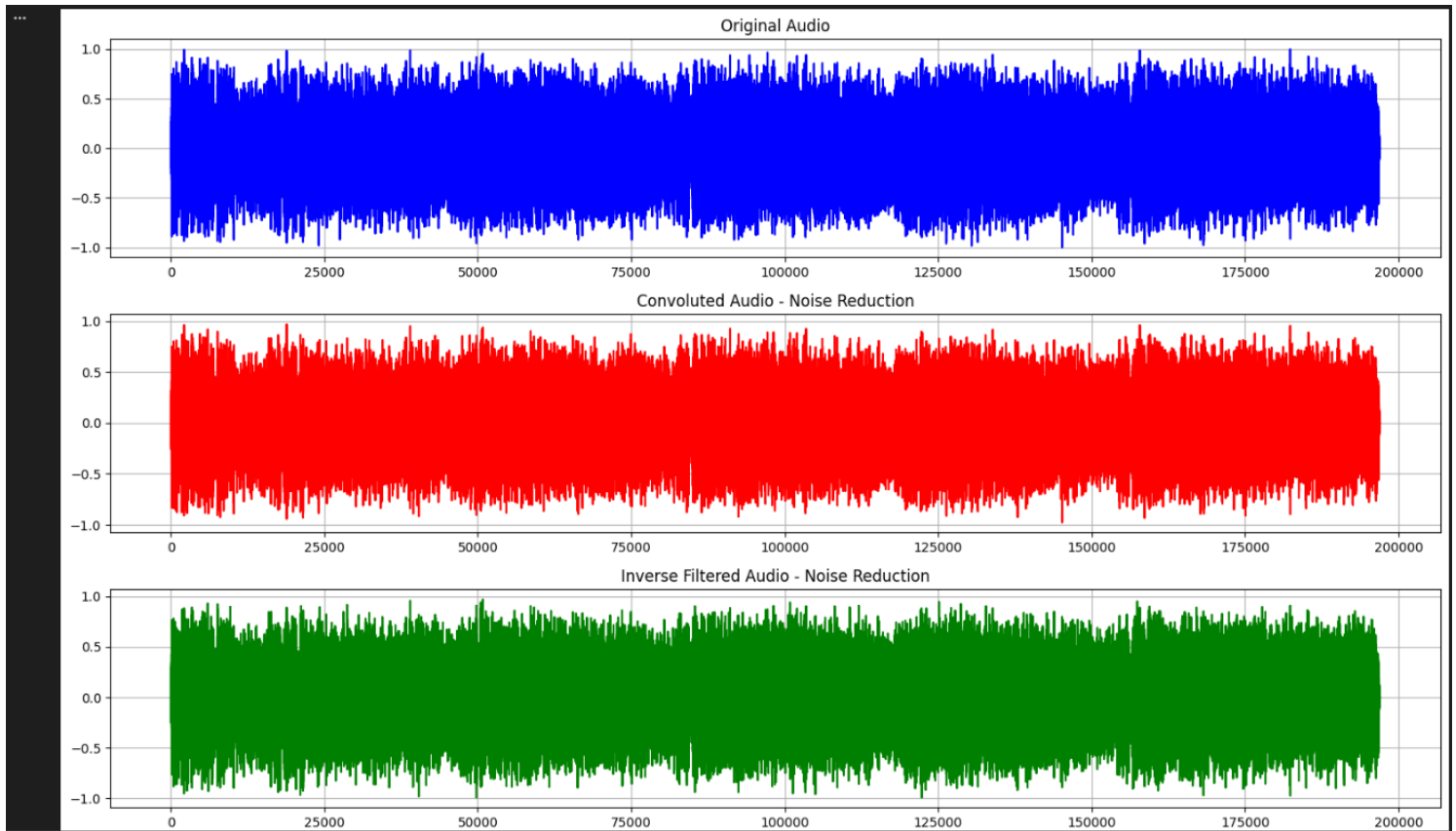

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147



... Noise Reduction Processing Completed

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "LowPass"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "LowPass_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "LowPass_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Low Pass Filter")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Low Pass Filter")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "LowPass.png"
    )
)

plt.show()

print("Low Pass Filter Processing Completed")
```

Subject: Digital Signal and Image Processing(01CT1513)

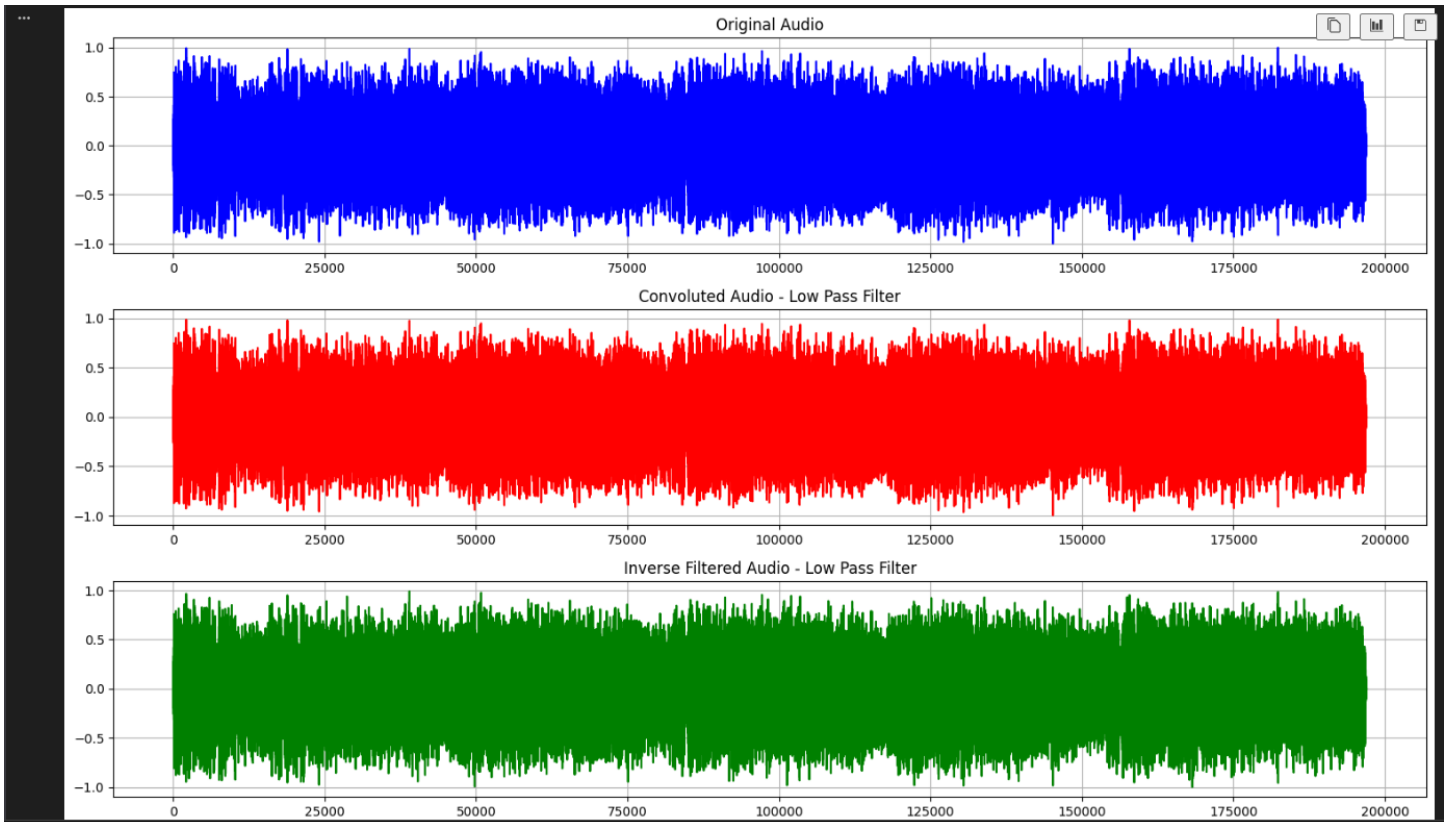
Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
... C:\Users\devah\AppData\Local\Temp\ipykernel_15888\1564470523.py:15: RuntimeWarning: divide by zero encountered in divide
1 / H,
C:\Users\devah\AppData\Local\Temp\ipykernel_15888\1564470523.py:15: RuntimeWarning: invalid value encountered in divide
1 / H,
```



```
... Low Pass Filter Processing Completed
```

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "HighPass"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "HighPass_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "HighPass_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - High Pass Filter")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - High Pass Filter")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "HighPass.png"
    )
)

plt.show()

print("High Pass Filter Processing Completed")
```

Subject: Digital Signal and Image Processing(01CT1513)

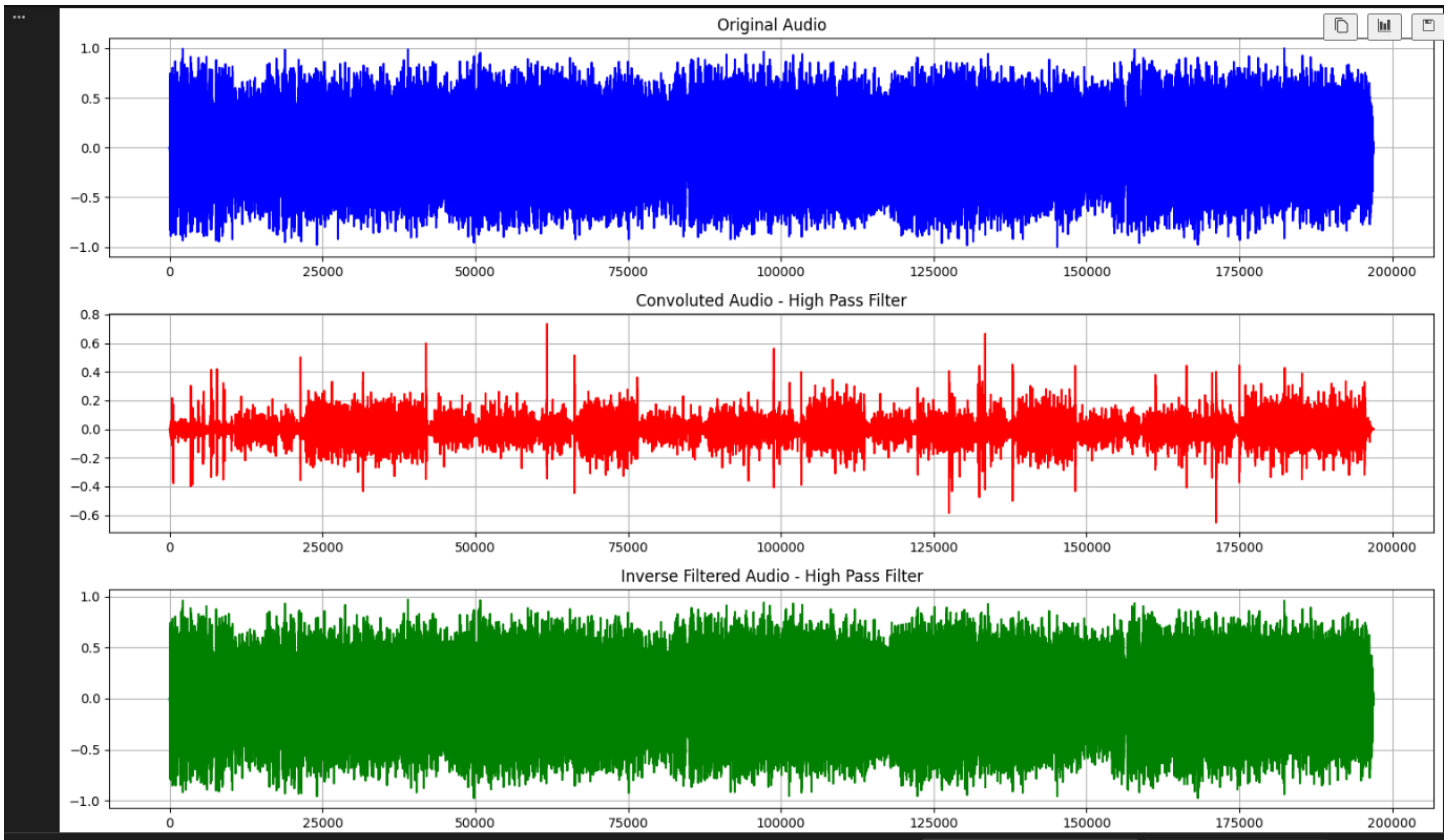
Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
... C:\Users\devah\AppData\Local\Temp\ipykernel_15080\3533428708.py:15: RuntimeWarning: divide by zero encountered in divide  
1 / H,  
C:\Users\devah\AppData\Local\Temp\ipykernel_15080\3533428708.py:15: RuntimeWarning: invalid value encountered in divide  
1 / H,
```



```
... High Pass Filter Processing Completed
```

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "EdgeDetection"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "EdgeDetection_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "EdgeDetection_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Edge Detection")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Edge Detection")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "EdgeDetection.png"
    )
)

plt.show()

print("Edge Detection Processing Completed")
```

Subject: Digital Signal and Image Processing(01CT1513)

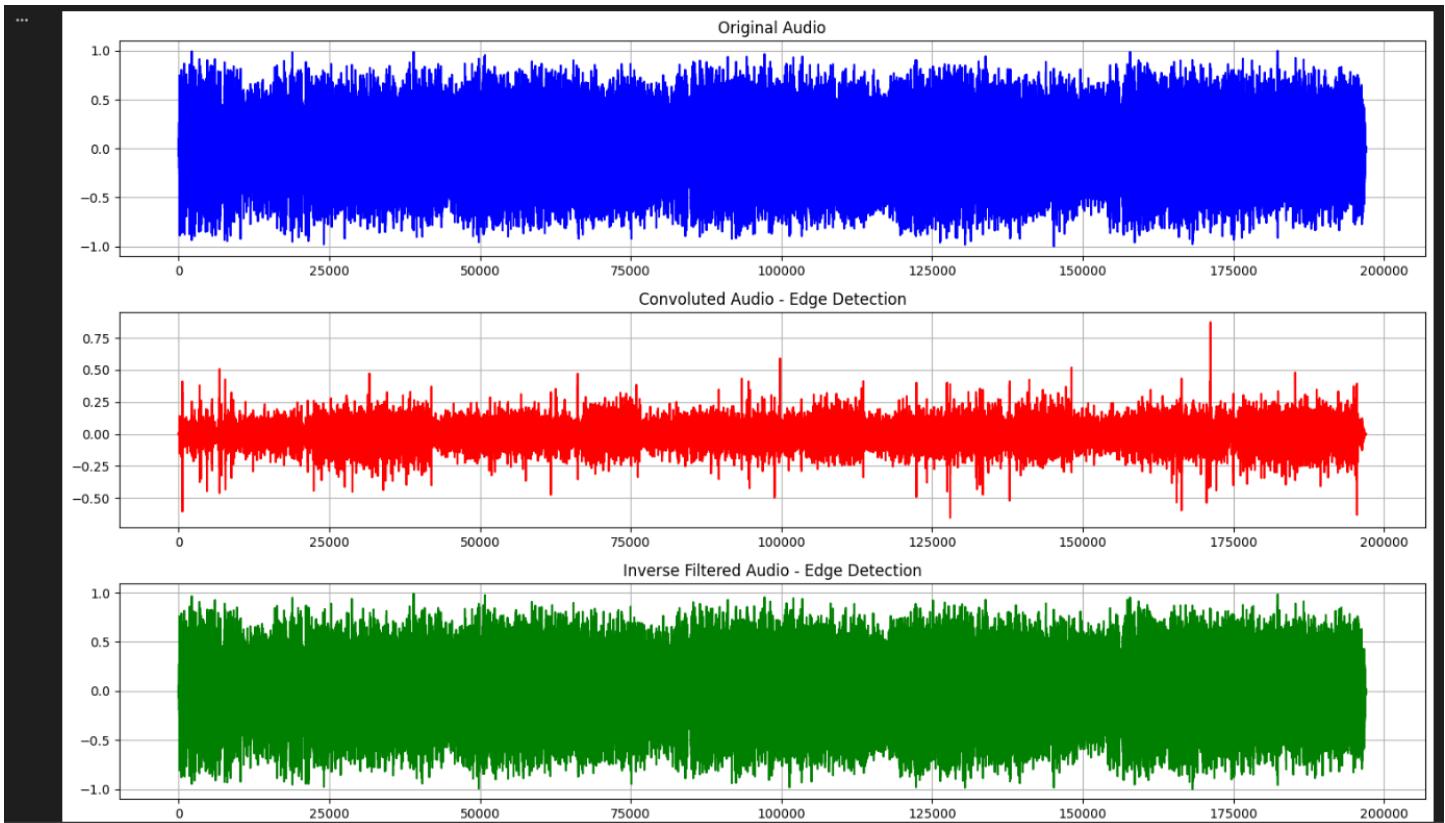
Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
*** C:\Users\devah\AppData\Local\Temp\ipykernel_15080\2110426607.py:15: RuntimeWarning: divide by zero encountered in divide
1 / H,
C:\Users\devah\AppData\Local\Temp\ipykernel_15080\2110426607.py:15: RuntimeWarning: invalid value encountered in divide
1 / H,
```



```
*** Edge Detection Processing Completed
```

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "Sharpen"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "Sharpen_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "Sharpen_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Sharpen")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Sharpen")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "Sharpen.png"
    )
)

plt.show()

print("Sharpen Processing Completed")
```

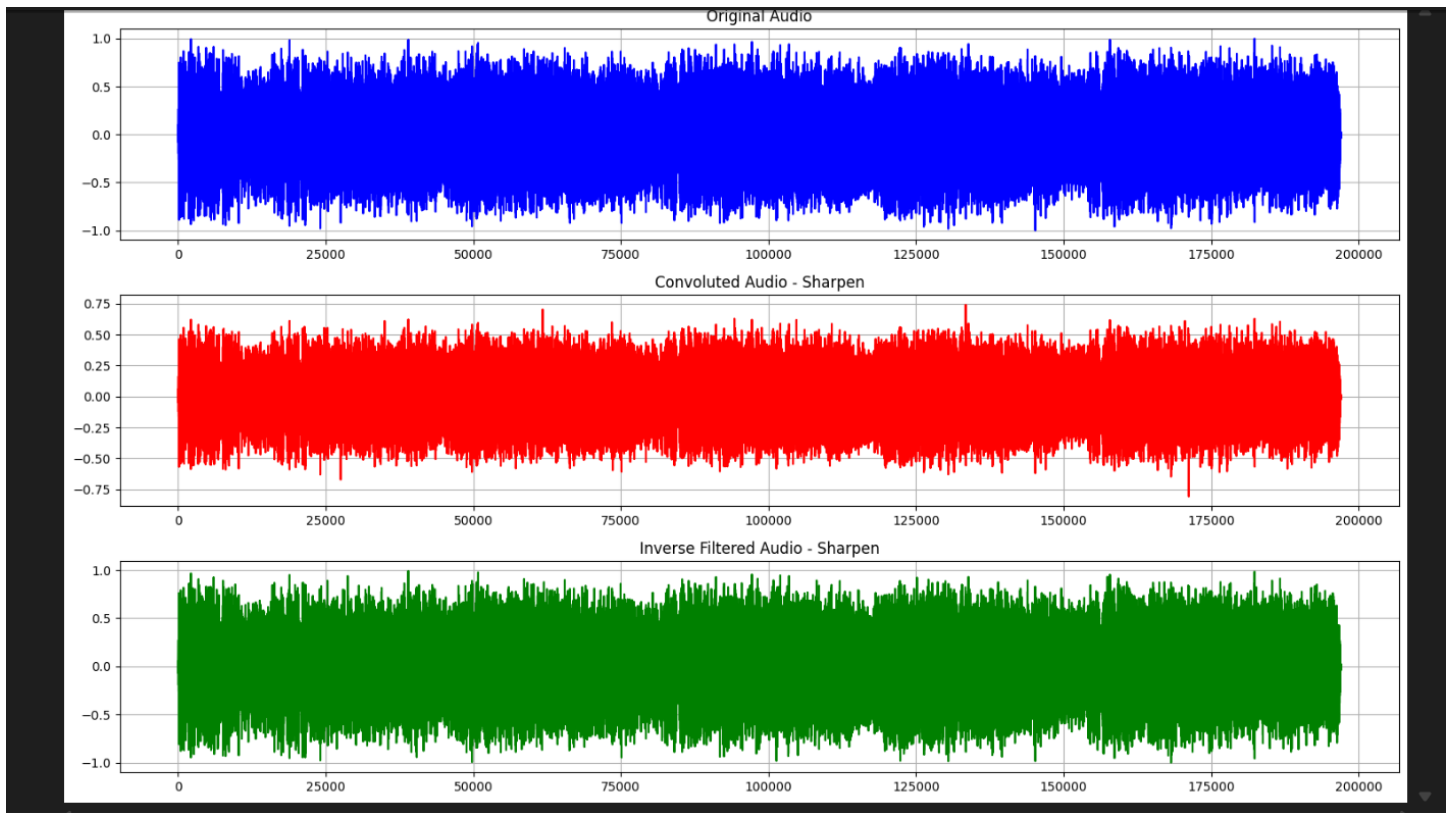

**Subject: Digital Signal and
Image Processing(01CT1513)**

**Aim: Audio Convolution and Inverse Filtering Using Different Impulse
Responses**

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147



... Sharpen Processing Completed

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```

name = "RoomReverb"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "RoomReverb_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "RoomReverb_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Room Reverb")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Room Reverb")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "RoomReverb.png"
    )
)

plt.show()

print("Room Reverb Processing Completed")

```

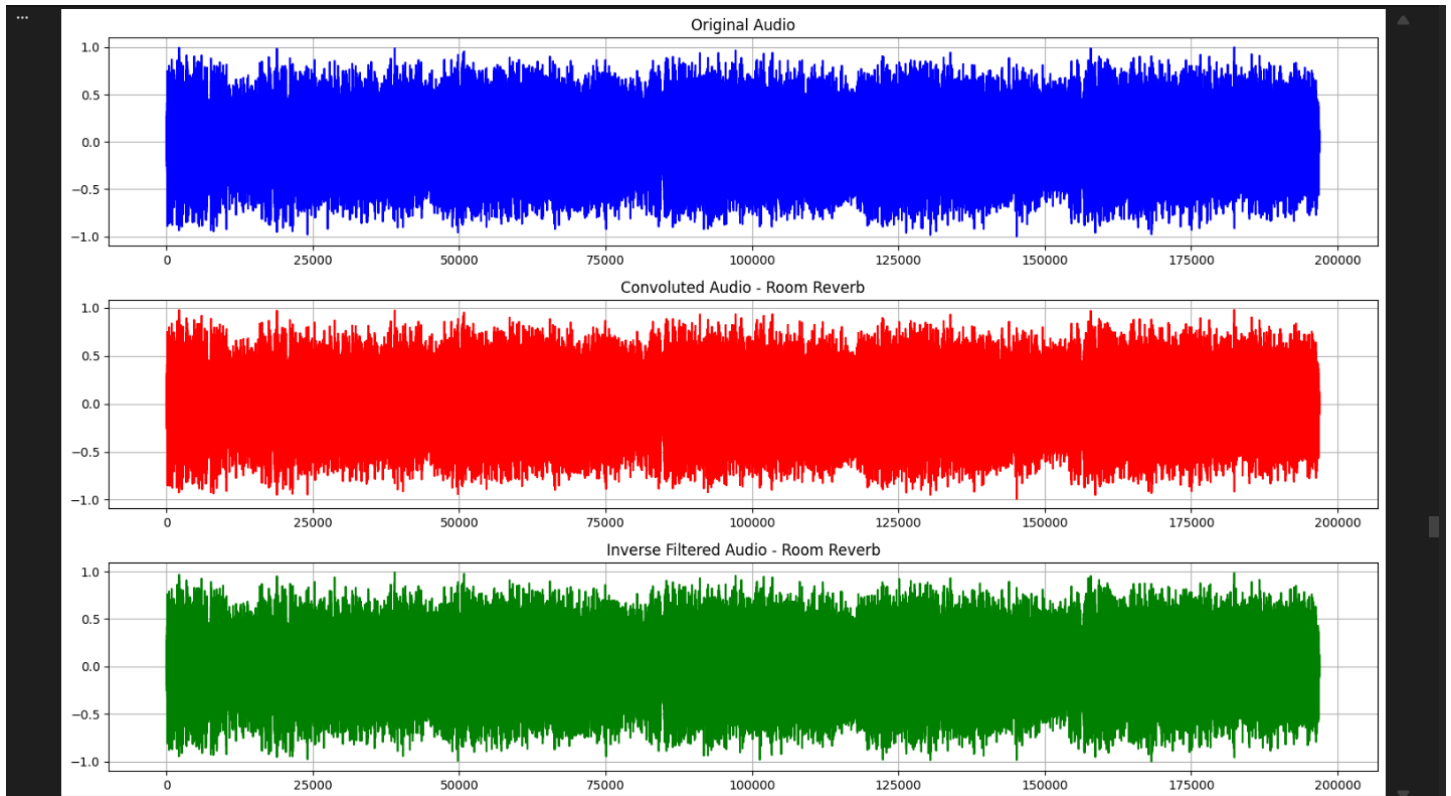
Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147



... Room Reverb Processing Completed

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "HallReverb"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "HallReverb_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "HallReverb_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Hall Reverb")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Hall Reverb")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "HallReverb.png"
    )
)

plt.show()

print("Hall Reverb Processing Completed")
```

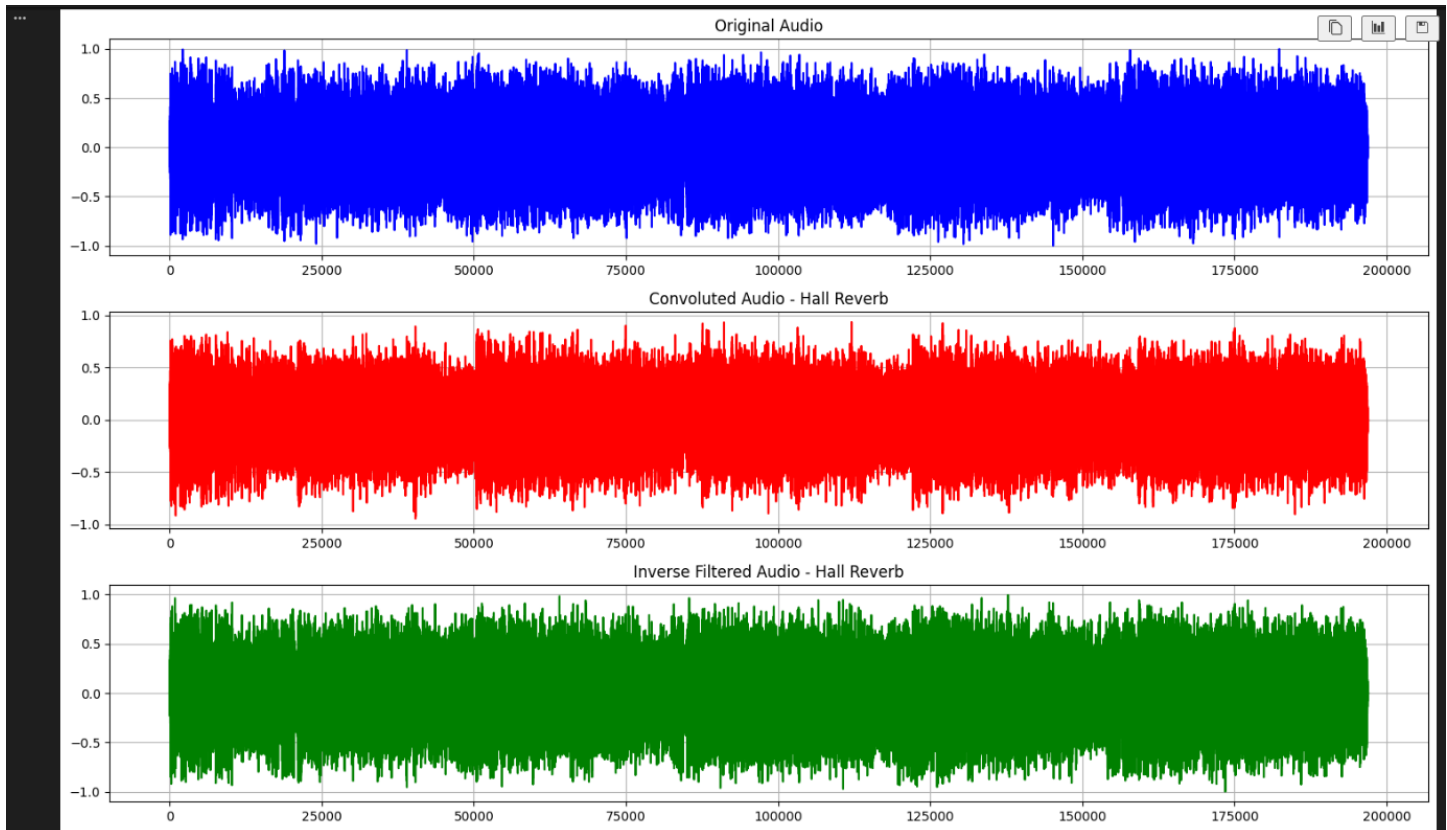
Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147



... Hall Reverb Processing Completed

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```
name = "Echo"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "Echo_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "Echo_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Echo")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Echo")
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(
        graph_folder,
        "Echo.png"
    )
)

plt.show()

print("Echo Processing Completed")
```

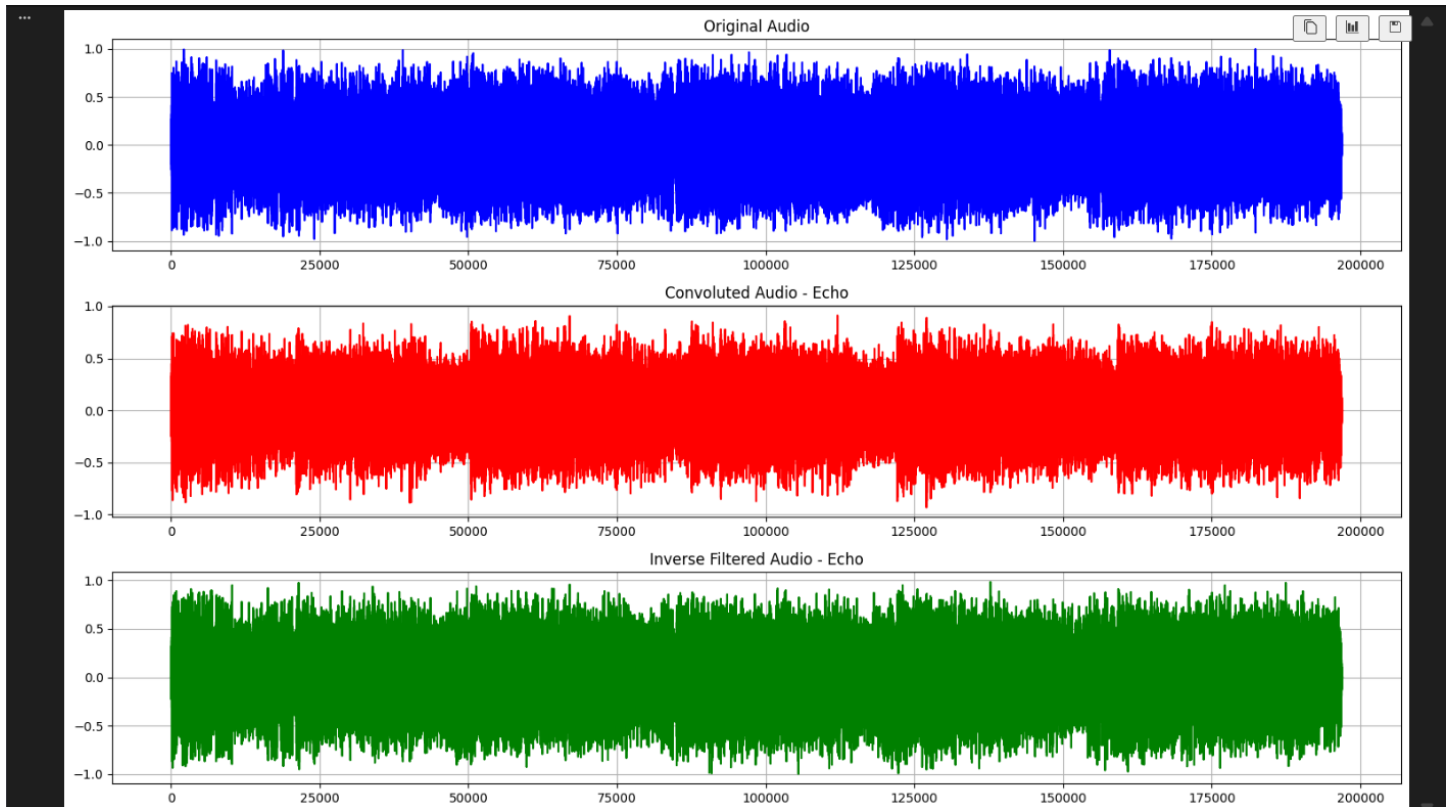
Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147



... Echo Processing Completed

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147

```

name = "Decay"

h = impulse_responses[name]

convolved = convolve(audio, h, mode="same")

convolved = convolved / np.max(np.abs(convolved))

H = fft(h, len(convolved))

epsilon = 1e-6

inverse_filter = np.where(
    np.abs(H) > epsilon,
    1 / H,
    0
)

restored = np.real(
    ifft(
        fft(convolved) * inverse_filter
    )
)

restored = restored / np.max(np.abs(restored))

wavfile.write(
    os.path.join(
        audio_folder,
        "Decay_Convolved.wav"
    ),
    fs,
    (convolved * 32767).astype(np.int16)
)

wavfile.write(
    os.path.join(
        audio_folder,
        "Decay_InverseFiltered.wav"
    ),
    fs,
    (restored * 32767).astype(np.int16)
)

plt.figure(figsize=(15,9))

plt.subplot(3,1,1)
plt.plot(audio[:100], color="blue")
plt.title("Original Audio")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(convolved[:100], color="red")
plt.title("Convolved Audio - Exponential Decay")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(restored[:100], color="green")
plt.title("Inverse Filtered Audio - Exponential Decay")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.savefig(
    os.path.join(
        graph_folder,
        "Decay.png"
    )
)
plt.show()
print("Exponential Decay Processing Completed")

```

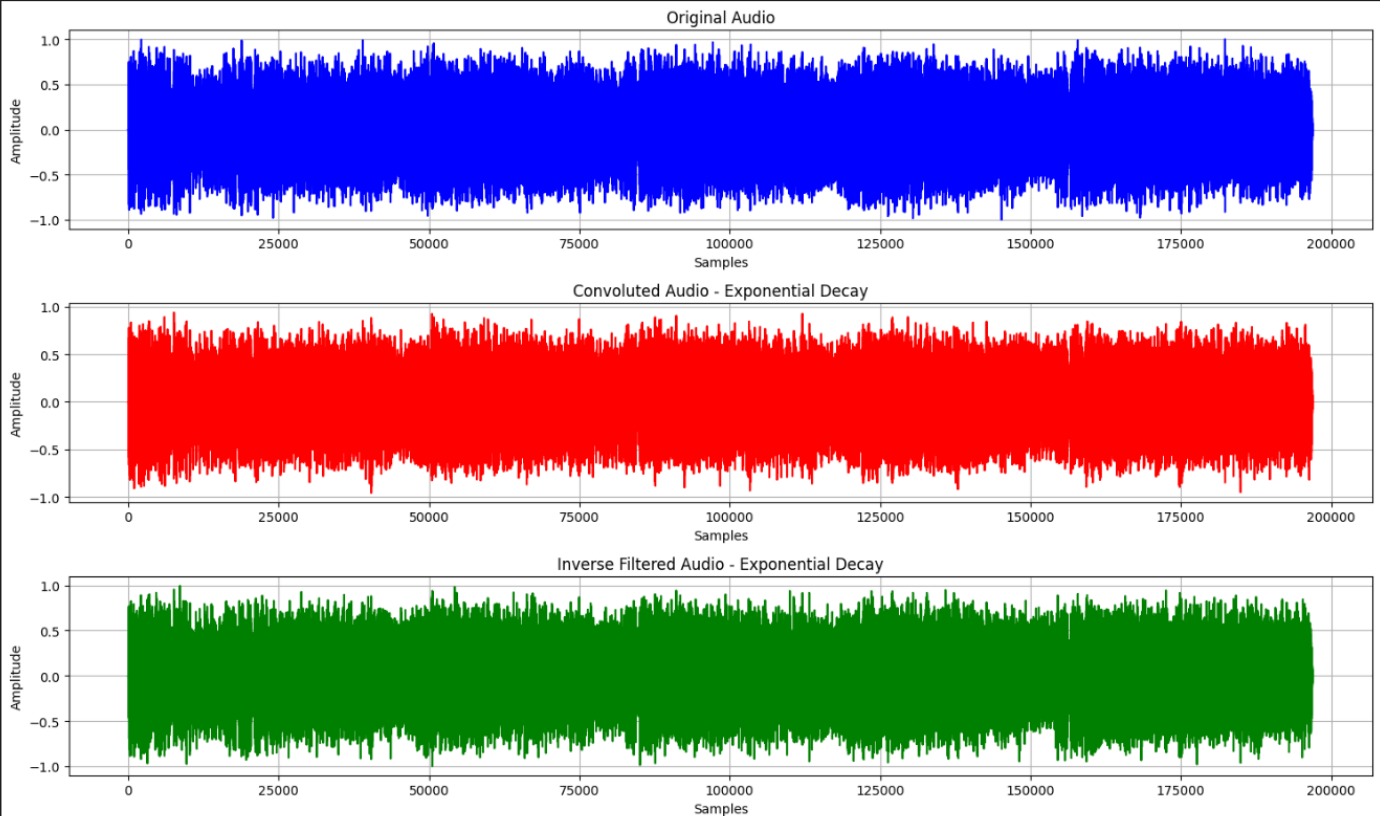

Subject: Digital Signal and Image Processing(01CT1513)

Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses

Open Ended Problem: 2

Date: 31/08/2026

Enrollment No: 92400133147



... Exponential Decay Processing Completed

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

```

print("AUDIO CONVOLUTION AND INVERSE FILTERING COMPLETED")

print("\nGenerated Audio Files\n")

for name in impulse_responses.keys():
    print(f"{name}_Convoluted.wav")
    print(f"{name}_InverseFiltered.wav")

print("\nGenerated Graphs\n")

print("Identity.png")
print("NoiseReduction.png")
print("LowPass.png")
print("HighPass.png")
print("EdgeDetection.png")
print("Sharpen.png")
print("RoomReverb.png")
print("HallReverb.png")
print("Echo.png")
print("Decay.png")

print("\nTotal Convoluted Audio Files      : 10")
print("Total Inverse Filtered Audio Files: 10")
print("Total Graphs                        : 10")

print("\nAudio Folder")

print(os.path.abspath(audio_folder))

print("\nGraph Folder")

print(os.path.abspath(graph_folder))

print("\nExperiment Completed Successfully")

```

[16] ✓ 0.0s Python

```

... AUDIO CONVOLUTION AND INVERSE FILTERING COMPLETED

Generated Audio Files

Identity_Convoluted.wav
Identity_InverseFiltered.wav
NoiseReduction_Convoluted.wav
NoiseReduction_InverseFiltered.wav
LowPass_Convoluted.wav
LowPass_InverseFiltered.wav
HighPass_Convoluted.wav
HighPass_InverseFiltered.wav
EdgeDetection_Convoluted.wav
EdgeDetection_InverseFiltered.wav
Sharpen_Convoluted.wav
Sharpen_InverseFiltered.wav
RoomReverb_Convoluted.wav
RoomReverb_InverseFiltered.wav
HallReverb_Convoluted.wav
HallReverb_InverseFiltered.wav
Echo_Convoluted.wav
Echo_InverseFiltered.wav
Decay_Convoluted.wav
Decay_InverseFiltered.wav
...
Graph Folder
c:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Graphs

Experiment Completed Successfully
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings..

```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

Output

The program generates:

- Original waveform
- 10 convolved audio files
- 10 inverse filtered audio files
- 10 waveform comparison graphs (Original, Convolved, Inverse Filtered)

Observations

Identity

The convolved and restored signals are almost identical to the original signal.

Noise Reduction

The moving average filter smooths the waveform and reduces background noise. The restored signal closely resembles the original.

Low-Pass Filter

High-frequency components are attenuated. Background music becomes smoother, but some speech details are also reduced.

High-Pass Filter

Low-frequency components are removed, making higher-frequency components more prominent. Restoration is only partial.

Edge Detection

Sudden waveform changes are emphasized. It is useful for detecting transients but not effective for background music removal.

Sharpen

Enhances waveform details and increases audio clarity, but may also amplify noise.

Room Reverb

Introduces mild reverberation. The inverse filter removes part of the reverberation but some artifacts remain.

Hall Reverb

Produces stronger reverberation with longer reflections. Restoration quality decreases due to overlapping echoes.

Echo

Delayed repetitions of the audio are introduced. Inverse filtering reduces the echo but cannot completely eliminate it.

Exponential Decay

Produces a gradual fading effect. The restored signal is improved but still contains residual reverberation.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT1513)	Aim: Audio Convolution and Inverse Filtering Using Different Impulse Responses	
Open Ended Problem: 2	Date: 31/08/2026	Enrollment No: 92400133147

Comparison Table

Impulse Response	Convolved Signal	Restored Signal	Effectiveness
Identity	No distortion	Excellent restoration	Excellent
Noise Reduction	Smooth waveform	Very good	Good
Low-Pass	High frequencies reduced	Good	Moderate
High-Pass	Low frequencies reduced	Moderate	Moderate
Edge Detection	Sharp transitions	Poor	Poor
Sharpen	Enhanced details	Moderate	Moderate
Room Reverb	Mild reverberation	Fair	Limited
Hall Reverb	Strong reverberation	Fair	Limited
Echo	Delayed repetitions	Partial	Limited
Exponential Decay	Gradual fading	Partial	Limited

Result

The program successfully applied convolution and inverse filtering using ten different impulse responses. The convolved and restored audio files, along with the waveform comparison graphs, were generated successfully. The experiment demonstrated that different impulse responses produce different effects on the audio signal and that inverse filtering can partially recover the original signal depending on the characteristics of the impulse response.

Conclusion

The experiment successfully demonstrated the application of convolution and inverse filtering in digital audio processing. Simple impulse responses such as Identity and Noise Reduction allowed the inverse filter to recover the original signal with high accuracy. More complex impulse responses such as Room Reverb, Hall Reverb, Echo, and Exponential Decay introduced reverberation and delayed reflections that could not be completely removed by inverse filtering. The experiment highlights that inverse filtering is effective only when the system response is accurately known and well-conditioned, making it useful for controlled environments but limited for real-world audio restoration involving strong reverberation or complex background music.

GitHub:

<https://github.com/Devaharshagoud/DSIP-Experiments.git>